IC10/instructions

From Stationeers Community Wiki

See IC10 for the primary page for the IC10 instruction set. This page lists all available instructions

Utility

```
alias str r?|d?
```

§

Labels register or device reference with name, device references also affect what shows on the screws on the IC base.

Example:

```
alias dAutoHydro1 d0
alias vTemperature r0
```

```
define str num
```

§

Creates a label that will be replaced throughout the program with the provided value.

Example:

```
define ultimateAnswer 42
move r0 ultimateAnswer # Store 42 in register 0
```

```
hcf
```

Halt and catch fire

```
sleep a(r?|num)
```

Pauses execution on the IC for a seconds

```
yield
```

Pauses execution for 1 tick

Mathematical

```
abs r? a(r?|num)
```

Register = the absolute value of a

Example:

```
define negativeNumber -10
abs r0 negativeNumber # Compute the absolute value of -10 and store it in register 0
```

```
add r? a(r?|num) b(r?|num)
```

§

Register = a + b.

Example:

```
add r0 r0 1 # increment r0 by one
```

```
define num1 10
define num2 20
add r0 num1 num2 # Add 10 and 20 and store the result in register 0
```

```
ceil r? a(r?|num)
```

§

Register = smallest integer greater than a

Example:

```
define floatNumber 10.3
ceil r0 floatNumber # Compute the ceiling of 10.3 and store it in register 0
```

```
div r? a(r?|num) b(r?|num)
```

§

Register = a / b

```
pow r? a(r?|num) b(r?|num)
```

§

Stores the result of raising a to the power of b in the register. Follows IEEE-754 standard for floating point arithmetic.

```
exp r? a(r?|num)
```

§

exp(a) or e^a

```
floor r? a(r?|num)
```

§

Register = largest integer less than a

```
log r? a(r?|num)
```

§

base e log(a) or ln(a)

```
max r? a(r?|num) b(r?|num)
```

§

Register = max of a or b

```
min r? a(r?|num) b(r?|num)
```

§

Register = min of a or b

```
mod r? a(r?|num) b(r?|num)
```

§

Register = a mod b (note: NOT a % b)

Example:

```
mod r0 10 20 # Expected: r0=10
mod r1 22 20 # Expected: r1 = 2
mod r2 22 -20 # Expected: r2 = 18
mod r2 22 -10 # Expected: r2 = 18
mod r2 -7 4 # Expected: r2 = 1
mod r2 -7 9 # Expected: r2 = 2
```

```
move r? a(r?|num)
```

§

Register = provided num or register value.

Example:

```
move r0 42 # Store 42 in register 0
```

```
mul r? a(r?|num) b(r?|num)
```

§

Register = a * b

```
rand r?
```

§

Register = a random value x with 0 <= x < 1

```
round r? a(r?|num)
```

§

Register = a rounded to nearest integer

```
sqrt r? a(r?|num)
```

§

Register = square root of a

```
sub r? a(r?|num) b(r?|num)
```

§

Register = a - b.

```
trunc r? a(r?|num)
```

§

Register = a with fractional part removed

```
lerp r? a(r?|num) b(r?|num) c(r?|num)
```

§

Linearly interpolates between a and b by the ratio c, and places the result in the register provided. The ratio c will be clamped between 0 and 1.

Mathematical / Trigonometric

```
acos r? a(r?|num)
```

§

Returns the angle (radians) whos cos is the specified value

```
asin r? a(r?|num)
```

§

Returns the angle (radians) whos sine is the specified value

```
atan r? a(r?|num)
```

§

Returns the angle (radians) whos tan is the specified value

```
atan2 r? a(r?|num) b(r?|num)
```

§

Returns the angle (radians) whose tangent is the quotient of two specified values: a (y) and b (x)

```
cos r? a(r?|num)
```

§

Returns the cosine of the specified angle (radians)

```
sin r? a(r?|num)
```

§

Returns the sine of the specified angle (radians)

```
tan r? a(r?|num)
```

§

Returns the tan of the specified angle (radians)

Stack

```
clr d?
```

§

Clears the stack memory for the provided device.

```
clrd id(r?|num)
```

§

Seeks directly for the provided device id and clears the stack memory of that device

```
get r? device(d?|r?|id) address(r?|num)
```

§

Using the provided device, attempts to read the stack value at the provided address, and places it in the register.

```
getd r? id(r?|id) address(r?|num)
```

§

Seeks directly for the provided device id, attempts to read the stack value at the provided address, and places it in the register.

```
peek r?
```

§

Register = the value at the top of the stack

```
poke address(r?|num) value(r?|num)
```

§

Stores the provided value at the provided address in the stack.

```
pop r?
```

§

Register = the value at the top of the stack and decrements sp

```
push a(r?|num)
```

§

Pushes the value of a to the stack at sp and increments sp

```
put device(d?|r?|id) address(r?|num) value(r?|num)
```

§

Using the provided device, attempts to write the provided value to the stack at the provided address.

```
putd id(r?|id) address(r?|num) value(r?|num)
```

§

Seeks directly for the provided device id, attempts to write the provided value to the stack at the provided address.

Slot/Logic

```
l r? device(d?|r?|id) logicType
```

§

Loads device LogicType to register by housing index value.

Example:

Read from the device on d0 into register 0

```
l r0 d0 Setting
```

Read the pressure from a sensor

```
l r1 d5 Pressure
```

This also works with aliases. For example:

```
alias Sensor d0
l r0 Sensor Temperature
```

```
lr r? device(d?|r?|id) reagentMode int
```

§

Loads reagent of device's ReagentMode where a hash of the reagent type to check for. ReagentMode can be either Contents (0), Required (1), Recipe (2). Can use either the word, or the number.

```
ls r? device(d?|r?|id) slotIndex logicSlotType
```

§

Loads slot LogicSlotType on device to register.

Example:

Read from the second slot of device on d0, stores 1 in r0 if it's occupied, 0 otherwise.

```
ls r0 d0 2 Occupied
```

And here is the code to read the charge of an AIMeE:

```
alias robot d0
alias charge r10
ls charge robot 0 Charge
```

```
s device(d?|r?|id) logicType r?
```

§

Stores register value to LogicType on device by housing index value.

Example:

```
s d0 Setting r0
```

```
ss device(d?|r?|id) slotIndex logicSlotType r?
```

§

Stores register value to device stored in a slot LogicSlotType on device.

```
rmap r? d? reagentHash(r?|num)
```

§

Given a reagent hash (signed 32bit int), store the corresponding prefab hash that the device expects to fulfill the reagent requirement. For example, on an autolathe, the hash for Iron will store the hash for ItemIronIngot.

Slot/Logic / Batched

```
lb r? deviceHash logicType batchMode
```

§

Loads LogicType from all output network devices with provided type hash using the provide batch mode. Average (0), Sum (1), Minimum (2), Maximum (3). Can use either the word, or the number.

Example:

```
lb r0 HASH("StructureWallLight") On Sum
```

```
lbn r? deviceHash nameHash logicType batchMode
```

§

Loads LogicType from all output network devices with provided type and name hashes using the provide batch mode. Average (0), Sum (1), Minimum (2), Maximum (3). Can use either the word, or the number.

```
lbns r? deviceHash nameHash slotIndex logicSlotType batchMode
```

§

Loads LogicSlotType from slotIndex from all output network devices with provided type and name hashes using the provide batch mode. Average (0), Sum (1), Minimum (2), Maximum (3). Can use

either the word, or the number.

```
lbs r? deviceHash slotIndex logicSlotType batchMode
```

Loads LogicSlotType from slotIndex from all output network devices with provided type hash using the provide batch mode. Average (0), Sum (1), Minimum (2), Maximum (3). Can use either the word, or the number.

```
sb deviceHash logicType r?
```

Stores register value to LogicType on all output network devices with provided type hash.

Example:

```
sb HASH("StructureWallLight") On 1
```

```
sbn deviceHash nameHash logicType r?
```

Stores register value to LogicType on all output network devices with provided type hash and name.

```
sbs deviceHash slotIndex logicSlotType r?
```

Stores register value to LogicSlotType on all output network devices with provided type hash in the provided slot.

Bitwise

```
and r? a(r?|num) b(r?|num)
```

Performs a bitwise logical AND operation on the binary representation of two values. Each bit of the result is determined by evaluating the corresponding bits of the input values. If both bits are 1, the resulting bit is set to 1. Otherwise the resulting bit is set to 0.

```
nor r? a(r?|num) b(r?|num)
```

Performs a bitwise logical NOR (NOT OR) operation on the binary representation of two values. Each bit of the result is determined by evaluating the corresponding bits of the input values. If both bits are 0, the resulting bit is set to 1. Otherwise, if at least one bit is 1, the resulting bit is set to 0.

```
not r? a(r?|num)
```

Performs a bitwise logical NOT operation flipping each bit of the input value, resulting in a binary complement. If a bit is 1, it becomes 0, and if a bit is 0, it becomes 1.

Note:

This is a bitwise operation, the NOT of 1 => -2, etc. You may want to use seqz instead

```
or r? a(r?|num) b(r?|num)
```

§

Performs a bitwise logical OR operation on the binary representation of two values. Each bit of the result is determined by evaluating the corresponding bits of the input values. If either bit is 1, the resulting bit is set to 1. If both bits are 0, the resulting bit is set to 0.

```
sla r? a(r?|num) b(r?|num)
```

§

Performs a bitwise arithmetic left shift operation on the binary representation of a value. It shifts the bits to the left and fills the vacated rightmost bits with zeros (note that this is indistinguishable from 'sll').

```
sll r? a(r?|num) b(r?|num)
```

§

Performs a bitwise logical left shift operation on the binary representation of a value. It shifts the bits to the left and fills the vacated rightmost bits with zeros.

```
sra r? a(r?|num) b(r?|num)
```

§

Performs a bitwise arithmetic right shift operation on the binary representation of a value. It shifts the bits to the right and fills the vacated leftmost bits with a copy of the sign bit (the most significant bit).

```
srl r? a(r?|num) b(r?|num)
```

§

Performs a bitwise logical right shift operation on the binary representation of a value. It shifts the bits to the right and fills the vacated leftmost bits with zeros

```
xor r? a(r?|num) b(r?|num)
```

§

Performs a bitwise logical XOR (exclusive OR) operation on the binary representation of two values. Each bit of the result is determined by evaluating the corresponding bits of the input values. If the bits are different (one bit is 0 and the other is 1), the resulting bit is set to 1. If the bits are the same (both 0 or both 1), the resulting bit is set to 0.

```
ext r? source(r?|num) offset(r?|num) length(r?|num)
```

§

Extracts a bit field from source value, beginning at bit offset for length bits and places result in the provided register. Payload cannot exceed 53 bits in final length.

```
ins r? field(r?|num) offset(r?|num) length(r?|num)
```

Inserts a bit field into the provided register, beginning at bit offset for length bits. Payload cannot exceed 53 bits in final length. NOTE: As of 2026-01-12, stable version has a bug with parameter order (uses offset-length-field instead of field-offset-length). Beta version has correct order.

Example:

```
move r0 $DE0000EF
move r1 $ADBE
ins r0 r1 8 16 #inserts field r1 at bit 8 for 16 bits, result: $DEADBEEF
```

Comparison

```
select r? a(r?|num) b(r?|num) c(r?|num)
```

Register = b if a is non-zero, otherwise c

Note:

This operation can be used as a simple ternary condition

Example:

1)
move r0 0
select r1 r0 10 200

```
move r0 0
select r1 r0 10 200
```

after run, r1 = 200

2)

move r0 5

select r1 r0 10 200

```
move r0 1
select r1 r0 10 100
```

after run, r1 = 10

Comparison / Device Pin

```
sdns r? device(d?|r?|id)
```

§

Register = 1 if device is not set, otherwise 0

```
sdse r? device(d?|r?|id)
```

§

Register = 1 if device is set, otherwise 0.

Comparison / Value

```
sap r? a(r?|num) b(r?|num) c(r?|num)
```

§

Register = 1 if abs(a - b) <= max(c * max(abs(a), abs(b)), float.epsilon * 8), otherwise 0

Example:

Set register to 1 if a and b are close enough to each other with the scaling factor of c. Equivalent to Python math.isclose

```
sapz r? a(r?|num) b(r?|num)
```

§

Register = 1 if abs(a) <= max(b * abs(a), float.epsilon * 8), otherwise 0

```
seq r? a(r?|num) b(r?|num)
```

§

Register = 1 if a == b, otherwise 0

```
seqz r? a(r?|num)
```

§

Register = 1 if a == 0, otherwise 0

```
sge r? a(r?|num) b(r?|num)
```

§

Register = 1 if a >= b, otherwise 0

```
sgez r? a(r?|num)
```

§

Register = 1 if a >= 0, otherwise 0

```
sgt r? a(r?|num) b(r?|num)
```

§

Register = 1 if a > b, otherwise 0

```
sgtz r? a(r?|num)
```

§

Register = 1 if a > 0, otherwise 0

```
sle r? a(r?|num) b(r?|num)
```

§

Register = 1 if a <= b, otherwise 0

```
slez r? a(r?|num)
```

§

Register = 1 if a <= 0, otherwise 0

```
slt r? a(r?|num) b(r?|num)
```

§

Register = 1 if a < b, otherwise 0

```
sltz r? a(r?|num)
```

§

Register = 1 if a < 0, otherwise 0

```
sna r? a(r?|num) b(r?|num) c(r?|num)
```

§

Register = 1 if abs(a - b) > max(c * max(abs(a), abs(b)), float.epsilon * 8), otherwise 0

```
snan r? a(r?|num)
```

§

Register = 1 if a is NaN, otherwise 0

```
snanz r? a(r?|num)
```

§

Register = 0 if a is NaN, otherwise 1

```
snaz r? a(r?|num) b(r?|num)
```

§

Register = 1 if abs(a) > max(b * abs(a), float.epsilon), otherwise 0

```
sne r? a(r?|num) b(r?|num)
```

Register = 1 if a != b, otherwise 0

```
snez r? a(r?|num)
```

Register = 1 if a != 0, otherwise 0

Branching

```
j int
```

Jump execution to line a

Example:

```
j 0 # jump line 0
```

```
j label # jump to a label

label:
# your code here
```

```
jal int
```

§

Jump execution to line a and store next line number in ra

Example:

jal provides a way to do function calls in IC10 mips

```
move r0 1000
move r1 0
start:
jal average
s db Setting r0
yield
j start

average:
add r0 r0 r1
div r0 r0 2
j ra # jump back
```

```
jr int
```

§

Relative jump to line a

Branching / Device Pin

```
bdnvl device(d?|r?|id) logicType a(r?|num)
```

§

Will branch to line a if the provided device not valid for a load instruction for the provided logic type.

```
bdnvs device(d?|r?|id) logicType a(r?|num)
```

§

Will branch to line a if the provided device not valid for a store instruction for the provided logic type.

```
bdns d? a(r?|num)
```

§

Branch to line a if device d isn't set

```
bdnsal d? a(r?|num)
```

§

Jump execution to line a and store next line number if device is not set

```
bdse d? a(r?|num)
```

§

Branch to line a if device d is set

```
bdseal d? a(r?|num)
```

§

Jump execution to line a and store next line number if device is set

Example:

```
#Store line number and jump to line 32 if d0 is assigned.
bdseal d0 32
```

```
#Store line in ra and jump to label HarvestCrop if device d0 is assigned.
bdseal d0 HarvestCrop
```

```
brdns d? a(r?|num)
```

§

Relative branch to line a if device is not set

```
brdse d? a(r?|num)
```

§

Relative branch to line a if device is set

Branching / Comparison

```
bap a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Branch to line d if abs(a - b) <= max(c * max(abs(a), abs(b)), float.epsilon * 8)

Example:

Branch if a and b are close enough to each other with the scaling factor of c. Equivalent to Python math.isclose

```
brap a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Relative branch to line d if abs(a - b) <= max(c * max(abs(a), abs(b)), float.epsilon * 8)

```
bapal a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Branch to line c if a != b and store next line number in ra

```
bapz a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if abs(a) <= max(b * abs(a), float.epsilon * 8)

```
brapz a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if abs(a) <= max(b * abs(a), float.epsilon * 8)

```
bapzal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if abs(a) <= max(b * abs(a), float.epsilon * 8) and store next line number in ra

```
beq a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a == b

```
breq a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if a == b

```
beqal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a == b and store next line number in ra

```
beqz a(r?|num) b(r?|num)
```

§

Branch to line b if a == 0

```
breqz a(r?|num) b(r?|num)
```

§

Relative branch to line b if a == 0

```
beqzal a(r?|num) b(r?|num)
```

§

Branch to line b if a == 0 and store next line number in ra

```
bge a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a >= b

```
brge a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if a >= b

```
bgeal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a >= b and store next line number in ra

```
bgez a(r?|num) b(r?|num)
```

§

Branch to line b if a >= 0

```
brgez a(r?|num) b(r?|num)
```

§

Relative branch to line b if a >= 0

```
bgezal a(r?|num) b(r?|num)
```

§

Branch to line b if a >= 0 and store next line number in ra

```
bgt a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a > b

Example:

An example of a **Schmitt** trigger, turning on a device if the temperature is too low, and turning it off if it's too high and finally doing nothing if the temperature is within the desired range.

```
alias sensor d0
alias device d1

define mintemp 293.15
define maxtemp 298.15

start:
yield
l r0 sensor Temperature
# If the temperature < mintemp, turn on the device
blt r0 mintemp turnOn
# If the temperature > maxtemp, turn off the device
```

```
bgt r0 maxtemp turnOff
j start

turnOn:
s device On 1
j start
turnOff:
s device On 0
j start
```

```
brgt a(r?|num) b(r?|num) c(r?|num)
```

§

relative branch to line c if a > b

```
bgtal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a > b and store next line number in ra

```
bgtz a(r?|num) b(r?|num)
```

§

Branch to line b if a > 0

```
brgtz a(r?|num) b(r?|num)
```

§

Relative branch to line b if a > 0

```
bgtzal a(r?|num) b(r?|num)
```

§

Branch to line b if a > 0 and store next line number in ra

```
ble a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a <= b

```
brle a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if a <= b

```
bleal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a <= b and store next line number in ra

```
blez a(r?|num) b(r?|num)
```

§

Branch to line b if a <= 0

```
brlez a(r?|num) b(r?|num)
```

§

Relative branch to line b if a <= 0

```
blezal a(r?|num) b(r?|num)
```

§

Branch to line b if a <= 0 and store next line number in ra

```
blt a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a < b

Example:

An example of a **Schmitt** trigger, turning on a device if the temperature is too low, and turning it off if it's too high and finally doing nothing if the temperature is within the desired range.

```
alias sensor d0
alias device d1

define mintemp 293.15
define maxtemp 298.15

start:
yield
l r0 sensor Temperature
# If the temperature < mintemp, turn on the device
blt r0 mintemp turnOn
# If the temperature > maxtemp, turn off the device
bgt r0 maxtemp turnOff
j start

turnOn:
s device On 1
j start
turnOff:
s device On 0
j start
```

```
brlt a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if a < b

```
bltal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a < b and store next line number in ra

```
bltz a(r?|num) b(r?|num)
```

§

Branch to line b if a < 0

```
brltz a(r?|num) b(r?|num)
```

§

Relative branch to line b if a < 0

```
bltzal a(r?|num) b(r?|num)
```

§

Branch to line b if a < 0 and store next line number in ra

```
bna a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Branch to line d if abs(a - b) > max(c * max(abs(a), abs(b)), float.epsilon * 8)

```
brna a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Relative branch to line d if abs(a - b) > max(c * max(abs(a), abs(b)), float.epsilon * 8)

```
bnaal a(r?|num) b(r?|num) c(r?|num) d(r?|num)
```

§

Branch to line d if abs(a - b) <= max(c * max(abs(a), abs(b)), float.epsilon * 8) and store next line number in ra

```
bnan a(r?|num) b(r?|num)
```

§

Branch to line b if a is not a number (NaN)

```
brnan a(r?|num) b(r?|num)
```

§

Relative branch to line b if a is not a number (NaN)

```
bnaz a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if abs(a) > max (b * abs(a), float.epsilon * 8)

```
brnaz a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if abs(a) > max(b * abs(a), float.epsilon * 8)

```
bnazal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if abs(a) > max (b * abs(a), float.epsilon * 8) and store next line number in ra

```
bne a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a != b

```
brne a(r?|num) b(r?|num) c(r?|num)
```

§

Relative branch to line c if a != b

```
bneal a(r?|num) b(r?|num) c(r?|num)
```

§

Branch to line c if a != b and store next line number in ra

```
bnez a(r?|num) b(r?|num)
```

§

branch to line b if a != 0

```
brnez a(r?|num) b(r?|num)
```

§

Relative branch to line b if a != 0

```
bnezal a(r?|num) b(r?|num)
```

§

Branch to line b if a != 0 and store next line number in ra
